

1. Partial Class

A partial class allows one class to be divided into multiple files using the partial keyword. It is useful when a class becomes too large.

In my project, I first created BookingController.cs with all booking related APIs in one file. It included seat locking, booking creation, payment validation, ticket generation, and booking history methods.

At beginning it was okay, but after adding more APIs the file became too lengthy and harder to manage.

So later I understood partial class could improve code structure.

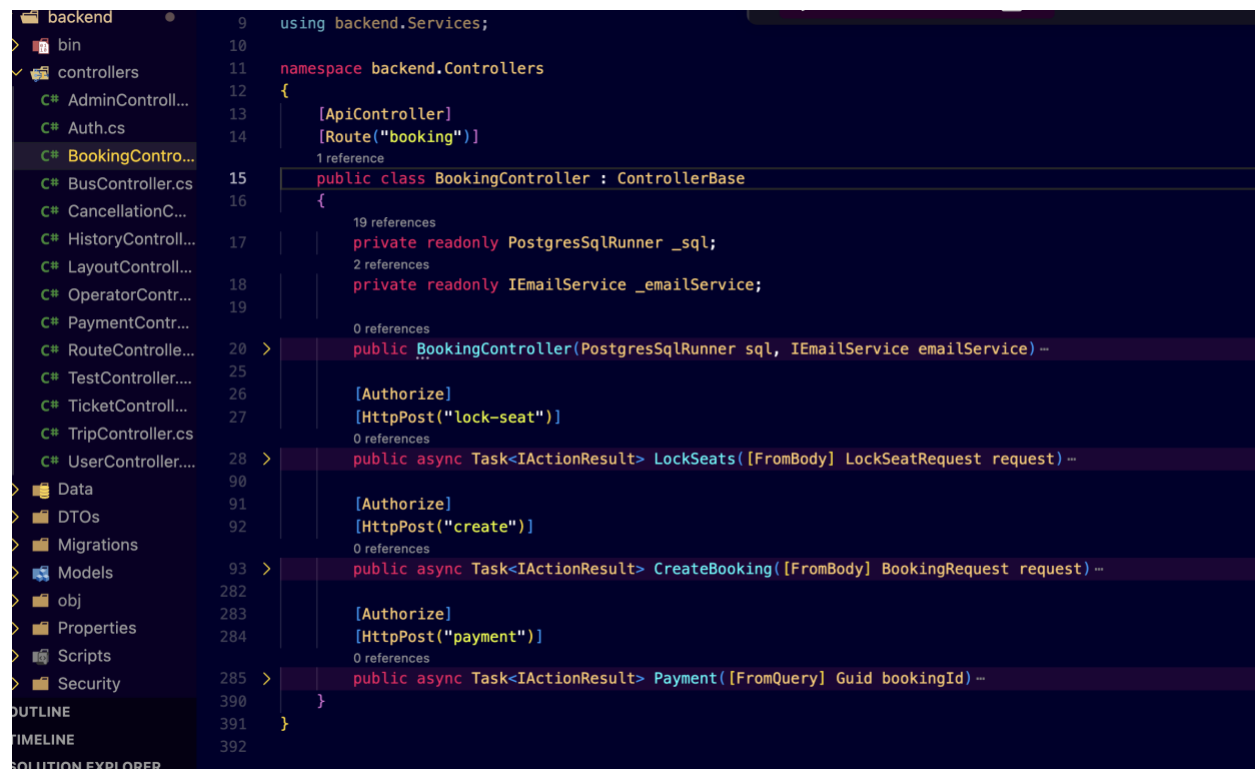
Before:

Single file:

File:

Controllers/BookingController.cs

All booking logic was inside one controller. (400 Lines of code)



The screenshot shows the Visual Studio IDE with the 'SOLUTION EXPLORER' on the left and the 'CODE' editor on the right. The 'SOLUTION EXPLORER' shows a project structure with folders like 'bin', 'controllers', 'Data', 'DTOs', 'Migrations', 'Models', 'obj', 'Properties', 'Scripts', and 'Security'. The 'controllers' folder is expanded, showing several controller files. The 'BookingController.cs' file is selected and its code is displayed in the 'CODE' editor. The code is a C# class named 'BookingController' that inherits from 'ControllerBase'. It has several methods: 'LockSeats', 'CreateBooking', and 'Payment'. The code is written in a dark theme with syntax highlighting. The line numbers are visible on the left side of the code editor.

```
9 using backend.Services;
10
11 namespace backend.Controllers
12 {
13     [ApiController]
14     [Route("booking")]
15     public class BookingController : ControllerBase
16     {
17         private readonly PostgreSQLRunner _sql;
18         private readonly IEmailService _emailService;
19
20         public BookingController(PostgreSQLRunner sql, IEmailService emailService) ...
21
22         [Authorize]
23         [HttpPost("lock-seat")]
24         public async Task<IActionResult> LockSeats([FromBody] LockSeatRequest request) ...
25
26         [Authorize]
27         [HttpPost("create")]
28         public async Task<IActionResult> CreateBooking([FromBody] BookingRequest request) ...
29
30         [Authorize]
31         [HttpPost("payment")]
32         public async Task<IActionResult> Payment([FromQuery] Guid bookingId) ...
33     }
34 }
```

After:

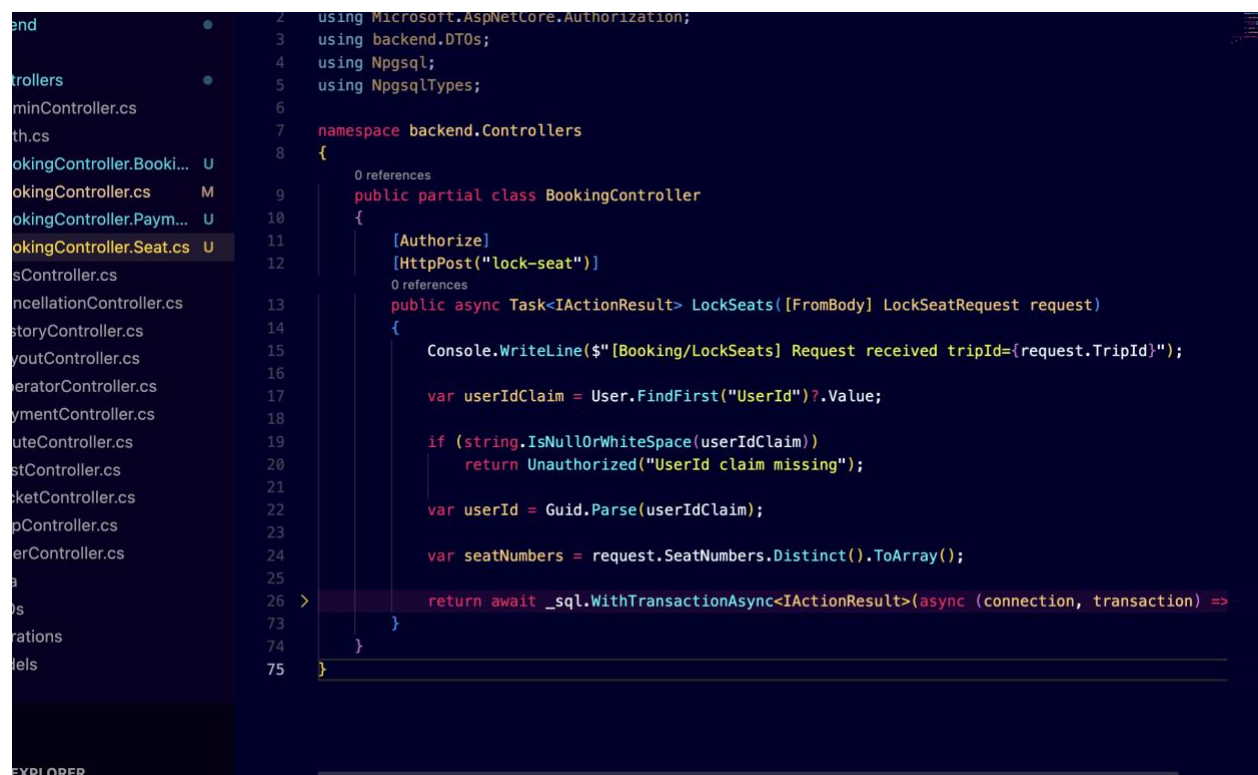
Split into smaller files:

Controllers/

- └─ BookingController.Seat.cs
- └─ BookingController.Payment.cs
- └─ BookingController.Booking.cs

Example:

File: BookingController.Seat.cs



```
2 using Microsoft.AspNetCore.Authorization;
3 using backend.DTOS;
4 using Npgsql;
5 using NpgsqlTypes;
6
7 namespace backend.Controllers
8 {
9     0 references
10    public partial class BookingController
11    {
12        [Authorize]
13        [HttpPost("lock-seat")]
14        0 references
15        public async Task<IActionResult> LockSeats([FromBody] LockSeatRequest request)
16        {
17            Console.WriteLine($"[Booking/LockSeats] Request received tripId={request.TripId}");
18
19            var userIdClaim = User.FindFirst("UserId")?.Value;
20
21            if (string.IsNullOrEmpty(userIdClaim))
22                return Unauthorized("UserId claim missing");
23
24            var userId = Guid.Parse(userIdClaim);
25
26            var seatNumbers = request.SeatNumbers.Distinct().ToArray();
27
28            return await _sql.WithTransactionAsync<IActionResult>(async (connection, transaction) =>
29            {
30            })
31        }
32    }
33 }
```

File: BookingController.Payment.cs

```
end
2 using Microsoft.AspNetCore.Authorization;
3 using Npgsql;
4
5 namespace backend.Controllers
6 {
7     0 references
8     public partial class BookingController
9     {
10         [Authorize]
11         [HttpPost("payment")]
12         0 references
13         public async Task<IActionResult> Payment([FromQuery] Guid bookingId)
14         {
15             Console.WriteLine($"[Booking/Payment] Payment started");
16
17             var userIdClaim = User.FindFirst("UserId")?.Value;
18
19             if (string.IsNullOrEmpty(userIdClaim))
20                 return Unauthorized("UserId claim missing");
21
22             var userId = Guid.Parse(userIdClaim);
23
24             > return await _sql.WithTransactionAsync<IActionResult>(async (connection, transaction) =>
25             {
26             }
27             )
28         }
29     }
30 }
```

File: BookingController.Booking.cs

```
end
5 using Npgsql;
6 using NpgsqlTypes;
7
8 namespace backend.Controllers
9 {
10     0 references
11     public partial class BookingController
12     {
13         [Authorize]
14         [HttpPost("create")]
15         0 references
16         public async Task<IActionResult> CreateBooking([FromBody] BookingRequest request)
17         {
18             Console.WriteLine($"[Booking/CreateBooking] Creating booking");
19
20             var userIdClaim = User.FindFirst("UserId")?.Value;
21
22             if (string.IsNullOrEmpty(userIdClaim))
23                 return Unauthorized("UserId claim missing");
24
25             var userId = Guid.Parse(userIdClaim);
26
27             var seatNumbers = request.Travelers
28                 .Select(x => x.SeatNumber)
29                 .Distinct()
30                 .ToArray();
31
32             > return await _sql.WithTransactionAsync<IActionResult>(async (connection, transaction) =>
33             {
34             }
35             )
36         }
37     }
38 }
```

2. Abstract Class

Abstract class helps when multiple classes share common behavior but implementation can be different.

In my project, notification system was one good place to use this.

Whenever booking gets confirmed or cancelled, system sends notifications.

Files:

Before:

Notification code was directly written in service classes.

After:

Created base abstraction.

File:

Services/MessageService.cs

```
public abstract class MessageService
{
    public abstract void SendMessage(string content);

    public void SaveLogs()
    {
        Console.WriteLine("Message logged");
    }
}
```

Derived class:

File:

Services/SmsService.cs

```
public class SmsService : MessageService
{
    public override void SendMessage(string content)
    {
        Console.WriteLine("SMS sent: " + content);
    }
}
```

3. Generic Class

Generic classes helped me reduce repeated code.

In my project, many APIs returned same structure:

- success status
- response message
- actual data

Instead of creating many response classes, I used one generic model.

Before:

Files like:

BusResult.cs
RouteResult.cs
UserResult.cs

After:

File:

Common/ResultModel.cs

```
public class ResultModel<T>
{
    public bool Success { get; set; }
    public string Message { get; set; }
    public T Result { get; set; }
}
```

Usage:

```
var response = new ResultModel<User>()
{
    Success = true,
    Message = "User fetched",
    Result = user
};
```

4. Repository Pattern

Repository pattern helped separate database code from controllers.

At first, I wrote SQL queries directly inside TripController.cs.

It worked, but controller became crowded.

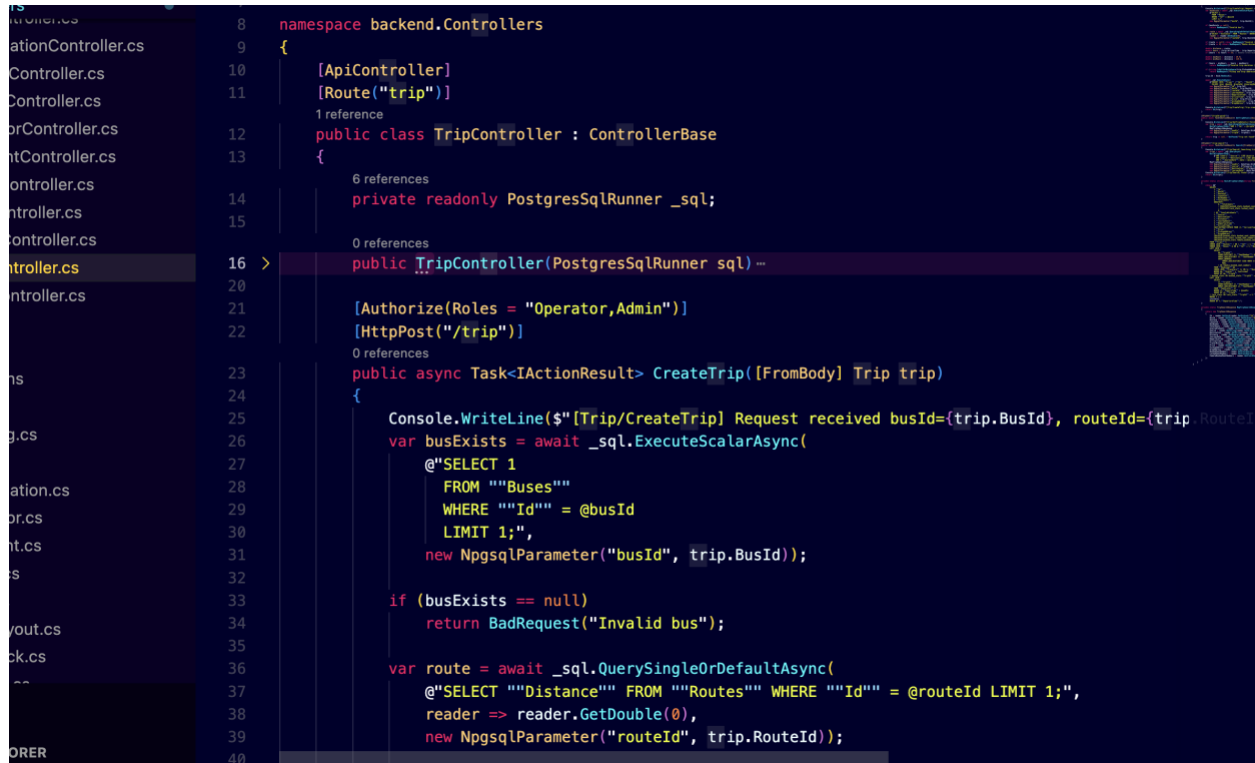
Later I moved DB logic into repository classes.

Before:

File:

Controllers/TripController.cs

Database logic inside controller.



```
8 namespace backend.Controllers
9 {
10     [ApiController]
11     [Route("trip")]
12     public class TripController : ControllerBase
13     {
14         private readonly PostgreSQLRunner _sql;
15
16         public TripController(PostgreSQLRunner sql) ~
17         {
18             [Authorize(Roles = "Operator,Admin")]
19             [HttpPost("/trip")]
20
21             public async Task<IActionResult> CreateTrip([FromBody] Trip trip)
22             {
23                 Console.WriteLine($"[Trip/CreateTrip] Request received busId={trip.BusId}, routeId={trip.RouteId}");
24                 var busExists = await _sql.ExecuteScalarAsync(
25                     @"SELECT 1
26                     FROM ""Buses""
27                     WHERE ""Id"" = @busId
28                     LIMIT 1;",
29                     new NpgsqlParameter("busId", trip.BusId));
30
31                 if (busExists == null)
32                     return BadRequest("Invalid bus");
33
34                 var route = await _sql.QuerySingleOrDefaultAsync(
35                     @"SELECT ""Distance"" FROM ""Routes"" WHERE ""Id"" = @routeId LIMIT 1;",
36                     reader => reader.GetDouble(0),
37                     new NpgsqlParameter("routeId", trip.RouteId));
38             }
39         }
40     }
```

After:

Files:

Controllers/TripController.cs

```

3 using backend.Models;
4 using backend.Repositories.Interfaces;
5
6 namespace backend.Controllers
7 {
8     [ApiController]
9     public class TripController : ControllerBase
10     {
11         private readonly ITripRepository _tripRepository;
12
13         public TripController(ITripRepository tripRepository)
14         {
15             _tripRepository = tripRepository;
16         }
17
18         [Authorize(Roles = "Operator,Admin")]
19         [HttpPost("/trip")]
20         public async Task<IActionResult> CreateTrip([FromBody] Trip trip) --
21
22         [HttpGet("trip/{tripId}")]
23         public async Task<IActionResult> GetTripDetails(Guid tripId) --
24
25         [HttpGet("/trip-search")]
26         public async Task<IActionResult> Search(
27             DateTime date) --
28     }
29 }
30
31 // using Microsoft.AspNetCore.Mvc;

```

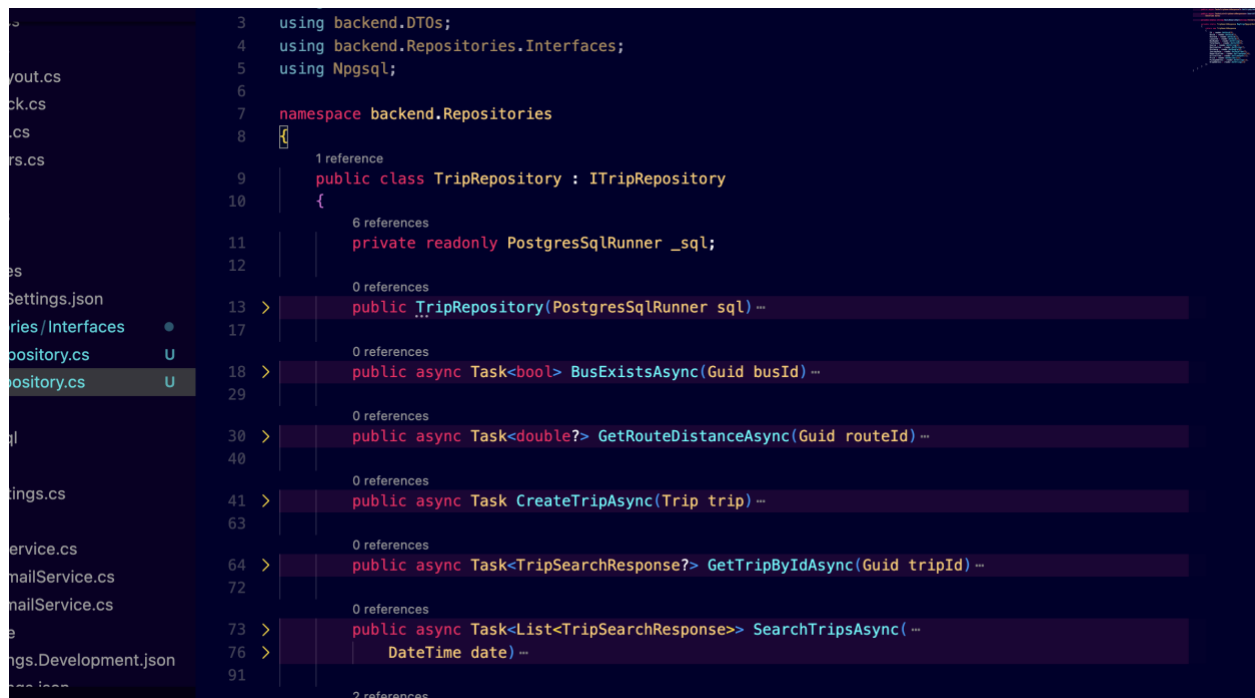
Repositories/Interfaces/ITripRepository.cs

```

3
4 namespace backend.Repositories.Interfaces
5 {
6     public interface ITripRepository
7     {
8         Task CreateTripAsync(Trip trip);
9
10        Task<bool> BusExistsAsync(Guid busId);
11
12        Task<double?> GetRouteDistanceAsync(Guid routeId);
13
14        Task<TripSearchResponse?> GetTripByIdAsync(Guid tripId);
15
16        Task<List<TripSearchResponse>> SearchTripsAsync(
17            string source,
18            string destination,
19            DateTime date);
20    }
21 }

```

Repositories/TripRepository.cs



```
3 using backend.DTOS;
4 using backend.Repositories.Interfaces;
5 using Npgsql;
6
7 namespace backend.Repositories
8 {
9     1 reference
10     public class TripRepository : ITripRepository
11     {
12         6 references
13         private readonly PostgreSQLRunner _sql;
14
15         0 references
16         public TripRepository(PostgreSQLRunner sql) --
17
18         0 references
19         public async Task<bool> BusExistsAsync(Guid busId) --
20
21         0 references
22         public async Task<double?> GetRouteDistanceAsync(Guid routeId) --
23
24         0 references
25         public async Task CreateTripAsync(Trip trip) --
26
27         0 references
28         public async Task<TripSearchResponse?> GetTripByIdAsync(Guid tripId) --
29
30         0 references
31         public async Task<List<TripSearchResponse>> SearchTripsAsync(DateTime date) --
32
33         2 references
34     }
```

5. Model Preparation

Model design was very important because all modules were connected.

Entities included:

- Customer
- Bus
- Route
- Journey
- Reservation
- Passenger

Initially some models had extra unnecessary fields, and some DTOs were mixed with entities.

Later I organized them properly.

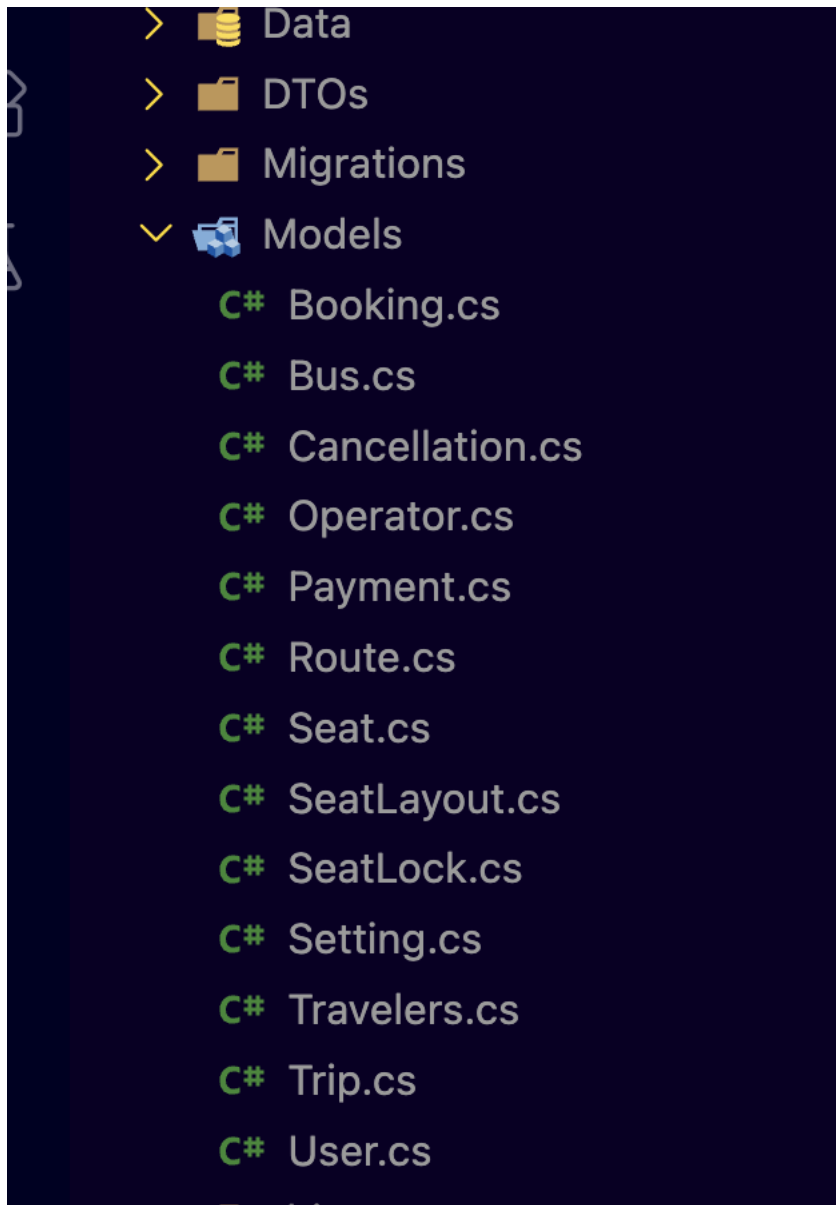
Before:

Mixed classes inside one folder.

After:

Models/
DTOs/
ViewModels/

Example:



6. Interface (Extra Design Improvement)

Interfaces helped in dependency injection.

Example in payment service.

Files:

Services/IPaymentGateway.cs
Services/DummyPaymentGateway.cs

Example:

```
public interface IPaymentGateway
{
    bool Process(decimal amount);
}
public class DummyPaymentGateway : IPaymentGateway
{
    public bool Process(decimal amount)
    {
        return true;
    }
}
```